7. El atributo [SerializeField]

[SerializeField] es uno de los atributos más utilizados en Unity. Permite exponer una variable privada o protegida en el Inspector sin necesidad de hacerla pública, mejorando la seguridad y la organización del código.

En este proyecto aparece en prácticamente todos los scripts: **Jugador**, **Bala**, **Enemigo**, **Humano**, **Explosion**, **GameManager**, etc.

7.1. ¿Qué hace exactamente [SerializeField]?

Permite que Unity **serialice** la variable para que aparezca en el Inspector, incluso si es privada o protegida.

Ejemplo del script **Jugador**:

```
[SerializeField] float numeroDecimal = 0, velocidad = 3f;

[SerializeField] int vida = 5;

[SerializeField] string nombre = "PJ 1";

[SerializeField] bool invulnerable = false;

[SerializeField] Vector3 posicionInicial = new Vector3(0, 0, 0);

[SerializeField] int[] municiones = { 500, 0, 0 };
```

Sin [SerializeField], estas variables no serían visibles ni editables desde el Editor.



7.2. Diferencia entre public y [SerializeField]

Opción	Visible en Inspector	Accesible desde otros scripts	Recomendado
public	Sí	Sí	Solo para acceso real desde otra clase
[SerializeField] private	Sí	No	Mejor práctica habitual
private	No	No	Útil para valores internos

7.3. ¿Por qué evitar usar public para todo?

- Expone datos innecesariamente
- Permite que otros scripts modifiquen variables internas
- Aumenta errores y dificulta el mantenimiento
- Rompe el principio de "encapsulación" en programación

7.4. Ejemplos reales del proyecto

En el script Enemigo

[SerializeField] int vida = 3;

[SerializeField] float velocidad = 5, limIzquierdo = -6, limDerecho = 6, tiempoDisparo = 3;

[SerializeField] bool derecha = true;

[SerializeField] GameObject prefabExplosion, prefabBalaE;

Aquí se ajustan desde el Inspector:

- Velocidad del enemigo
- Límites de movimiento
- Prefabs que usa para disparar y explotar

En el script Humano

[SerializeField] int municion = 10, maxMunicion = 10;

[SerializeField] float velocidad = 5, fuerzaSalto = 6, distanciaSensorSuelo = 0.6f;

[SerializeField] bool enSuelo = false;

[SerializeField] LayerMask mascaraSuelo;

El equipo de diseño puede ajustar:

- Fuerza de salto
- Velocidad
- Rango del sensor de suelo
- Capa que detecta como “suelo”

7.5. Serializar referencias a objetos

[SerializeField] también permite arrastrar objetos del Editor a un script:

En el script **Humano**:

[SerializeField] GameObject prefabBala;

[SerializeField] Transform puntoDisparo;

[SerializeField] TMP_Text textMunicion;

[SerializeField] Image imageBarra;

En el script **Jugador**:

[SerializeField] GameObject prefabBala;

Esto hace posible vincular:

- Prefabs
- Objetos de UI
- Transforms
- AudioSources
- Materiales
- Sprites

7.6. Serializar arrays, listas y vectores

Unity serializa arrays como:

[SerializeField] int[] municiones = { 500, 0, 0 };

Esto permite cambiar sus valores o su tamaño desde el Inspector.

También serializa:

- Vector2
- Vector3
- Color
- Quaternion
- AnimationCurve
- Sprites
- Objetos personalizados marcados como [System.Serializable]

7.7. Buenas prácticas con [SerializeField]

- Usar variables privadas + [SerializeField]
- Agrupar variables por temática
- Usar Tooltips o Headers si hace falta ordenar

Ejemplo:

```
[Header("Movimiento")]
[SerializeField] float velocidad;


[Header("Disparo")]
[SerializeField] GameObject prefabBala;
[SerializeField] float cooldownDisparo;
```

7.8. ¿Qué no se debe serializar?

- Variables temporales
- Estados lógicos internos (isJumping, isAttacking)
- Contadores que deben resetearse cada partida
- Referencias creadas dinámicamente en tiempo de ejecución